TEXT TO NUMBERS

Vocabulary, One-Hot, Multi-Hot, Bag of Words and TF-IDF: how each one looks, what breaks, and what fixes it

1. Why do we convert text into numbers?

After preprocessing we hold clean words, but a machine learning model is mathematics: it adds, multiplies and compares numbers. It cannot do anything with the word "great". The job of **text vectorization** (also called text representation or feature extraction) is to turn every document into a list of numbers, called a vector, in a way that keeps useful information. The methods in these notes are a chain: each one is invented to cure the biggest weakness of the one before it. All of them start from the same ingredient, the **vocabulary**.

Raw text	Preprocessing	Vocabulary	One-Hot	Multi-Hot	Bag of Words	TF-IDF

DONE IN PREVIOUS NOTES — THESE NOTES: from words to vectors

2. Our running example (a tiny random corpus)

Three short movie reviews that have **already been through preprocessing** (lowercased, cleaned, stop words removed). A corpus is the whole collection of documents; here every document is one review. The same three reviews are used in every method, so you can compare the results side by side.

Doc	Cleaned review (tokens)	Tokens	Sentiment
D1	great movie great acting	4	positive
D2	boring movie bad acting	4	negative
D3	great story movie	3	positive

3. The methods, one by one

01 BUILDING THE VOCABULARY

the dictionary your model is allowed to read

What it is: The complete list of unique words (tokens) found in the training texts, where every word gets its own fixed ID number called an index. Think of it as the dictionary of the model: a word outside it is invisible.

How it is built: (a) Pool the cleaned tokens of every training document. (b) Remove duplicates so each word is kept once. (c) Sort alphabetically or by frequency and number the words from 0. (d) Optionally trim rare words.

Why it is required: Every method after this makes one column per vocabulary word, so **vocabulary size = length of every vector**. A fixed word list means position 3 always means "great" for every document, which is what lets a model compare reviews.

Where it is used: Every vectorizer, deep-learning tokenizers, word embeddings, language models and search-engine indexes.

VOCABULARY OF OUR CORPUS

Index	Word	Total count	In docs
0	acting	2	2
1	bad	1	1
2	boring	1	1
3	great	3	2
4	movie	3	3
5	story	1	1

11 tokens in total -> **6 unique words** -> vocabulary size = 6. Every vector built later will have exactly 6 numbers. "Total count" = how often the word occurs everywhere; "In docs" = how many different documents contain it.

1 POOL ALL TOKENS (11)	2 REMOVE DUPLICATES (6)	3 SORT A-Z	4 GIVE INDEXES
great movie great acting boring movie bad acting great story movie	great, movie, acting, boring, bad, story	acting, bad, boring, great, movie, story	acting 0, bad 1, boring 2, great 3, movie 4, story 5

WATCH OUT

1. Build on training data only. Using test data to make the list leaks information and gives falsely good scores.

2. Unseen words. A new word like "awesome" at prediction time is not in the list. It is ignored or mapped to a special UNK (unknown) entry.

3. Size explodes. The 50,000 IMDB reviews contain tens of thousands of unique words, so vectors become very long and mostly zeros.

4. Preprocessing decides size. Without lowercasing, "Great", "great" and "GREAT" are three entries. Stemming and stop-word removal shrink it.

MAY WORK NEXT: the same vocabulary feeds One-Hot, Multi-Hot, Bag of Words and TF-IDF, so any mistake here (a typo, a missed lowercase) travels into all of them.

PRACTICAL CONTROLS USED IN REAL PROJECTS

1. Minimum frequency. Drop words seen fewer than, say, 2 times. This removes typos and one-off names and shrinks the list.

2. Maximum size. Keep only the top 5,000 to 10,000 most frequent words to control memory and speed.

3. Special entries. Add UNK for unknown words and PAD for padding, so every input has a place to go.

4. N-grams. Also add word pairs such as "not good", so a little word order survives.

SPREADSHEET PICTURE: imagine the vocabulary as the **column headings** of a spreadsheet and every document as one **row**. The four methods that follow differ only in what they write into the cells: One-Hot writes a 1 per word (one row per word), Multi-Hot writes 0 or 1, Bag of Words writes a count, and TF-IDF writes an importance weight.

02 ONE-HOT ENCODING (OHE)

one word = one vector

What it is: Every vocabulary word becomes a vector as long as the vocabulary, filled with 0 everywhere except a single 1 at that word's index. A sentence becomes a stack of these vectors, one row per word, in the original order.

Why it is required: Plain index numbers (acting = 0 ... story = 5) would mislead the model into thinking "story" is bigger than "great". One-hot gives every word its own private column, so no word looks larger or closer than another.

Where it is used: Categorical features (city, colour, class label), the input of small neural networks and sequence models (RNN, LSTM), and teaching examples of how text becomes numbers.

How to read the table: Each row is one word of the sentence, in order. The single 1 sits under that word's own column, so the row for "great" has its 1 under "great". Stack the rows and you hold the whole review with its word order intact, which is why one-hot is fed to sequence models word by word.

D1 = "great movie great acting" -> 4 rows x 6 = 24 numbers

	acting	bad	boring	great	movie	story
great	0	0	0	1	0	0
movie	0	0	0	0	1	0
great	0	0	0	1	0	0
acting	1	0	0	0	0	0

D3 = "great story movie" -> 3 rows x 6 = 18 numbers

	acting	bad	boring	great	movie	story
great	0	0	0	1	0	0
story	0	0	0	0	0	1
movie	0	0	0	0	1	0

WHAT CAN GO WRONG

1. Different shapes. A 4-word review gives 4 rows, a 3-word review gives 3. Most models need one fixed-size input, so you must pad or cut every review to the same length.

2. Huge and mostly zeros. With a 50,000-word vocabulary each word is 50,000 numbers with a single 1. One 100-word review becomes 5,000,000 numbers, over 99.99% zeros.

3. No meaning. Any two different words are equally far apart: "great" is as unrelated to "good" as it is to "boring". Similar words share nothing.

4. Unseen words. "Awesome" has no column, so it cannot be encoded at all and its information is lost.

MAY WORK NEXT: Multi-hot encoding squeezes all word rows of a review into ONE fixed-length vector, which cures problems 1 and 2 for whole documents.

03 MULTI-HOT ENCODING

one document = one 0/1 vector

What it is: Instead of one row per word, we keep one single row per document. Each vocabulary position gets a 1 if the word appears in the document at least once, otherwise 0. It is the one-hot rows of the document merged together (any 1 wins).

Why it is required: Every review, short or long, now has the same length (6 here), so ordinary algorithms such as logistic regression, Naive Bayes and SVM can be trained, and documents can be compared directly. It answers the simple question: "which words are present?"

Where it is used: Keyword-based spam filters, multi-label tagging (a film can be both "comedy" and "drama"), recommender systems (which items a user has seen), short texts and search queries.

MULTI-HOT MATRIX (3 documents x 6 words)

	acting	bad	boring	great	movie	story
D1	1	0	0	1	1	0
D2	1	1	1	0	1	0
D3	0	0	0	1	1	1

D1 contains "great" twice, but its multi-hot value is just **1**. D1 and D3 both say great + movie, so they look similar.

WHAT CAN GO WRONG

1. Counts are lost. "great movie great acting" and "great movie acting" give the same vector. Strong repeated praise looks the same as a single mention.

2. Order is lost. "dog bites man" and "man bites dog" are identical. "Not good" and "good" differ only by the presence of one word.

3. All words weigh the same. The rare, revealing word "boring" counts exactly like the filler word "movie".

4. Still large. For a big vocabulary each document is still tens of thousands of positions, almost all zeros.

MAY WORK NEXT: Bag of Words replaces the yes/no by a count, so repeated words are noticed.

04 BAG OF WORDS (BOW)

count how often each word occurs

What it is: Like multi-hot, each document is one fixed-length row, but each cell stores **how many times** the word occurs. Putting all rows together gives a document-term matrix. It is called a "bag" because the words are tossed in without order.

Why it is required: Frequency carries meaning: a review that says "great" twice is more enthusiastic. Counts keep more information than 0/1, and the method is simple, quick and a strong baseline.

Where it is used: Sentiment analysis, spam detection, news and document classification, topic modelling, and as the first baseline before heavier models.

BAG-OF-WORDS MATRIX (counts)

	acting	bad	boring	great	movie	story
D1	1	0	0	2	1	0
D2	1	1	1	0	1	0
D3	0	0	0	1	1	1

Now D1 shows **2** for "great". Note "movie" has a count in all three reviews even though it separates nothing.

WHAT CAN GO WRONG

1. Common words dominate. A word that appears in every review gets a large count but tells us nothing, while a rare telling word looks small.

2. Order and grammar are lost. "Movie was not good" and "movie was good" differ by a single count, so negation is barely visible.

3. Long documents win. A long review has bigger counts than a short one, so comparing them is unfair unless we normalise.

4. No meaning, huge size. "Great" and "excellent" are separate columns, and big vocabularies mean wide, sparse tables.

MAY WORK NEXT: TF-IDF multiplies the count share by how rare the word is across documents, so distinctive words win.

05 TF-IDF

count share x rarity = importance

What it is: TF-IDF is a weighted Bag of Words. Each cell = **TF x IDF**. **TF** (term frequency) = how many times the word appears in this document divided by the total words in the document. **IDF** (inverse document frequency) = the natural log of (total documents divided by documents that contain the word).

Why it is required: A word that is frequent in one review but rare in the rest describes that review and scores high. A word found in every document scores zero. Dividing by document length also makes short and long reviews comparable.

Where it is used: Search engines and ranking, keyword extraction, document similarity, recommendation, text classification, spam filtering and summarization.

IDF OF EACH WORD (N = 3 documents)

Word	In docs	IDF
acting	2	0.405
bad	1	1.099
boring	1	1.099
great	2	0.405
movie	3	0.000
story	1	1.099

IDF = ln(3 / docs). "movie" is in all 3 reviews, so ln(3/3) = 0. Libraries add smoothing, so their numbers differ slightly but the ranking is the same.

TF-IDF MATRIX (3 documents x 6 words)

	acting	bad	boring	great	movie	story
D1	0.101	0	0	0.203	0	0
D2	0.101	0.275	0.275	0	0	0
D3	0	0	0	0.135	0	0.366

WORKED EXAMPLE: "great" in D1

TF = 2 / 4 = 0.5. IDF = ln(3/2) = 0.405. TF-IDF = 0.5 x 0.405 = **0.203**.

HOW TO READ IT

"movie" is 0 everywhere because every review has it. "story" (0.366) is the signature word of D3, "bad" and "boring" (0.275) mark D2, and "great" leads D1. Compare with Bag of Words, where "movie" looked important.

Still not solved: TF-IDF keeps no word order and no meaning ("great" and "excellent" remain unrelated columns), and vectors stay long and sparse. Word embeddings (Word2Vec, GloVe, BERT) are the next step for that.

4. The same review, four different views

Method	Vector or matrix for D1 = "great movie great acting" (column order: acting, bad, boring, great, movie, story)
One-Hot	`4 separate rows, one per word: [0 0 0 1 0 0] [0 0 0 0 1 0] [0 0 0 1 0 0] [1 0 0 0 0 0]`
Multi-Hot	`[1, 0, 0, 1, 1, 0] (great twice is still 1)`
Bag of Words	`[1, 0, 0, 2, 1, 0] (great twice is 2)`
TF-IDF	`[0.101, 0, 0, 0.203, 0, 0] (movie drops to 0, great is boosted by rarity)`

5. All methods side by side

Method	What one document becomes	Counts?	Order?	Weights importance?	Biggest problem
One-Hot	Matrix: one 0/1 row per word (shape changes with length)	No	Yes	No	Variable size, huge, no meaning
Multi-Hot	One fixed 0/1 vector	No	No	No	Repeats and order lost
Bag of Words	One fixed vector of counts	Yes	No	No	Common words dominate
TF-IDF	One fixed vector of weights	Yes (as share)	No	Yes	No order, no meaning

The chain of problems and fixes: each method is born from the weakness of the previous one.

One-Hot	Multi-Hot	Bag of Words	TF-IDF	Embeddings
one row per word	one vector per document	counts added	importance weights	meaning in numbers
problem: variable size	problem: counts lost	problem: common words win	problem: no meaning	next topic

6. Which one should I choose?

Situation	Good choice
Need only "is the word there?" (keywords, tags, short queries)	Multi-Hot
Quick baseline for sentiment or spam where repeats matter	Bag of Words
Search, ranking, keyword extraction, similarity, longer documents	TF-IDF
Word order and meaning matter (translation, chatbots)	Embeddings, sequence models

7. One-line revision

Vocabulary	The fixed word list; its size sets the length of every vector.
One-Hot	One word = one vector with a single 1. Keeps order, but shapes vary and there is no meaning.
Multi-Hot	One document = one 0/1 vector. Fixed size, but counts and order are lost.
Bag of Words	One document = word counts. Simple and strong, but common words dominate.
TF-IDF	Count share x rarity. Shows which words truly describe a document.